

Introduction to MATLAB

Chapter 4 : Functions and Optimization methods

Julia M. Schmidt ¹

Master Quantitative Economics
Master International Affairs and Development

September 2026

¹julia.schmidt[at]dauphine.psl.eu

Introduction

- The user may sometimes create a procedure that will be applied for multiple variables, potentially separately from the main script, and different from the built-in MATLAB functions. It is then interesting to create external functions.
- MATLAB is a very interesting tool to solve complicated numerical problems. We will see how optimisation methods work on MATLAB.
- Today's session will be divided in 2 parts:
 1. How to create external functions
 2. Understand optimization methods
- Bonus: How to do this in python (GeconPy)

Why using functions?

- Store and use the same code to accomplish specific task for multiple arguments.
- Works as the functions studied in mathematics: take one or many arguments in input and process them to return one or many output arguments.
- Can be created outside the main script and be called within the script. The functions can therefore be used by different scripts.

Create external functions

- Function files are .m files, as script files.
- To easily create a function file :
 - Right-click on the "Current folder" window
 - Select "New" then "Function"
 - A new file, named "untitled.m" will be created, inviting you to rename it. The name of the file will also automatically be the name of the function.

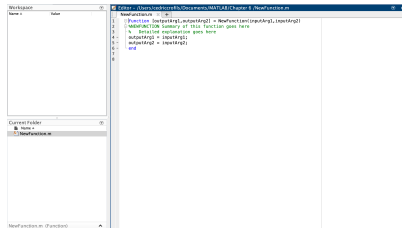


Figure: Creation of a new function

Create external functions

- The name of the function and the .m file should be the same. It is however possible to store multiples functions into a unique file.
- The name of the output arguments (outputArg1,outputArg2 in the previous figure) are the variables that will be returned when the process of the function is done. The last values of the variables will be returned.
- The name of the input arguments (inputArg1,inputArg2 in the figure) are the ones used throughout the code, i.e. the variables that will be used in the process of the function.

Example:

Let's create the function "Square", which calculate the square of a number.

```
function [xSquare] = Square(x)
% Calculate the square of a number
xSquare = x ^ 2;
end
```

N - input arguments functions

- We can include many variables as input arguments for the function.
- In the following example, we want to code the function "Power", which will return the n^{th} power of the given number.

Example:

```
function [powerX] = Power(x, n)
% Calculate the power n of a number
powerX = x ^ n;
end
```

N - output arguments functions

- Likewise, a function can return many variables as output arguments.
- The example will return the minimum, the maximum and the variance of a serie.

Example:

```
function [valMin, valMax, V] = stats(x)
% Calculate some statistics of a serie of numbers
valMin = min(x);
valMax = max(x);
V = var(x);
end
```

In the command window :

```
[value1, value2, value3] = stats(rand(20, 1))
```

Exercise 1

1. Create a function that calculates monthly, quarterly, or semi-annual equivalent rates, taking as input arguments the annual rate and the periodicity. Test your function for $r = 10\%$.
Hint : if a capital of 1 yields 1,10 at the end of a year, it yields $(1 + x)^{12}$, x being the monthly equivalent rate.
2. Create a function that returns both the log and the log difference of a serie as output. Test your function with a random vector of size 30.

Anonymous functions

- Instead of creating an external function, MATLAB enables the user to create an Anonymous function, in the workspace directly using the command window or written in the main script instead of creating a new file. The variable created is of type `function_handle`. The function will accept multiple inputs and return one output.
- To create an anonymous function, you need to use the `@` symbol.

Example:

```
sqr = @(x)x.^ 2;  
q = integral(sqr, 0, 1);  
q = integral(@(x)x.^ 2, 0, 1);
```

Modelling approaches

- Solver-Based Optimization Problem Setup
 - Choose the right optimization solver
 - Create the objective function and the potential constraint equations
 - Call the appropriate solver
- Problem-Based Optimization Setup
 - Create an Optimization problem
 - Define the problem variables and the objective function
 - Define the constraints
 - Use of the *solve* function to solve the problem

See more on MATLAB documentation : <https://fr.mathworks.com/help/optim/ug/first-choose-problem-based-or-solver-based-approach.html>

Solver-Based Optimization

Optimization Decision Table

The following table is designed to help you choose a solver. It does not address multiobjective optimization or equation solving. There are more details on all the solvers in [Problems Handled by Optimization Toolbox Functions](#).

In this table:

- * means relevant solvers are found in [Global Optimization Toolbox](#) (Global Optimization Toolbox) functions (licensed separately from Optimization Toolbox™ solvers).
- fmincon applies to most smooth objective functions with smooth constraints. It is not listed as a preferred solver for least squares or linear or quadratic programming because the listed solvers are usually more efficient.
- The table has suggested functions, but it is not meant to unduly restrict your choices. For example, fmincon can be effective on some nonsmooth problems.
- The Global Optimization Toolbox [ga](#) (Global Optimization Toolbox) and [surrogateopt](#) (Global Optimization Toolbox) functions can address mixed-integer nonlinear programming problems.
- The Statistics and Machine Learning Toolbox™ [bayesopt](#) (Statistics and Machine Learning Toolbox) function can address low-dimensional deterministic or stochastic optimization problems with combinations of continuous, integer, or categorical variables.

Solvers by Objective and Constraint

Constraint Type	Objective Type				
	Linear	Quadratic	Least Squares	Smooth Nonlinear	Nonsmooth
None	n/a ($f = \text{const. or min} = -\infty$)	quadprog, Information	mldivide, lsqcurvefit, lsqnonlin, Information	fminsearch, fminunc, Information	fminsearch, *
Bound	linprog, Information	quadprog, Information	lsqcurvefit, lsqlin, lsqnonlin, lsqnonneg, Information	fminbnd, fmincon, fseminf, Information	fminbnd, *
Linear	linprog, Information	quadprog, Information	lsqlin, Information	fmincon, fseminf, Information	*
Cone	coneprog, Information	fmincon, Information	fmincon, Information	fmincon, Information	*
General Smooth	fmincon, Information	fmincon, Information	fmincon, Information	fmincon, fseminf, Information	*
Discrete, with Bound or Linear	intlinprog, Information	*	*	*	*

Figure: Optimization decision table

Quadratic function

We want to solve the following optimization problem :

$$\min_{x \in [-5;5]} f(x) = 0.1x^2$$

To solve this problem on MATLAB using the solver-based approach, we define the function `fun1` and use the built-in function `fmincon()` to minimize it.

Quadratic function

```
% create a local function
fun1 = @(x).1 * x.^2;
X = -5 : 5;
% plot the function
plot(X,fun1(X))
x0 = 1;
[Xopt,fval] = fmincon(fun1,x0)
hold on;
plot(Xopt,fval,'o')
hold off;
```

Quadratic function

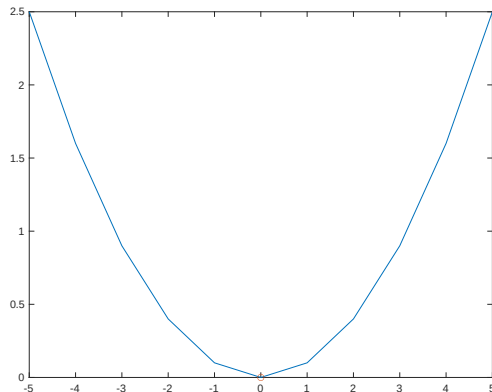


Figure: Solution of a quadratic function

Noisy quadratic function

We study now the previous optimization problem, augmented with some noise induced by the function sinus :

$$\min_{x \in [-5;5]} f(x) = 0.1x^2 + \sin(x)$$

We use the same method as before, and obtain the following code.

Noisy quadratic function

```
% create a local function
fun2 = @(x).1 * x.^2 + sin(x);
X = -5 : 5;
% plot the function
plot(X,fun2(X))
x0 = 1;
[Xopt, fval] = fmincon(fun2, x0)
hold on;
plot(Xopt,fval,'o')
Y = -5:0.01:5;
plot(Y,fun2(Y),'r-')
hold off;
```


Noisy quadratic function

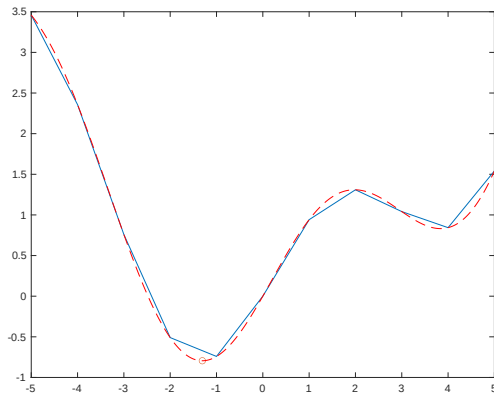


Figure: Solution of a noisy quadratic function

The starting guess is an input

Same function. Same solver. Same options. Change one arbitrary number:

x_0	x^*	f^*	
1	-1.3064	-0.7946	the global minimum
3	+3.8375	+0.8316	a local minimum

The two answers differ by 1.63, and **both report** `exitflag = 1` — success.

MATLAB is not lying. `fminunc` promises a *local* minimum and delivered one both times. The promise is simply weaker than everybody assumes.

⇒ *Plot the function. Try several starting points. If they disagree, the answer you report is a choice you made.*

Bounds restrictions

We want now to solve the previous problem, with the addition of two constraints on x :

$$\min_x f(x) = 0.1x^2 + \sin(x)$$

$$s.t. x > 2$$

$$s.t. x < 5$$

We still use the function `fmincon()` to solve it. You may see the documentation to see how to code it.

Bounds restrictions

```
% create a local function
fun2 = @(x).1 * x.^2 + sin(x);
X = -5 : 5;
% plot the function
plot(X,fun2(X))
x0 = 1;
[Xopt, fval] = fmincon(fun2, x0, [], [], [], [], 2, 5)
hold on;
plot(Xopt,fval,'o')
Y = -5:0.01:5;
plot(Y, fun2(Y),'r-')
xline(2,'k')
xline(5,'k')
hold off;
```

Bounds restrictions

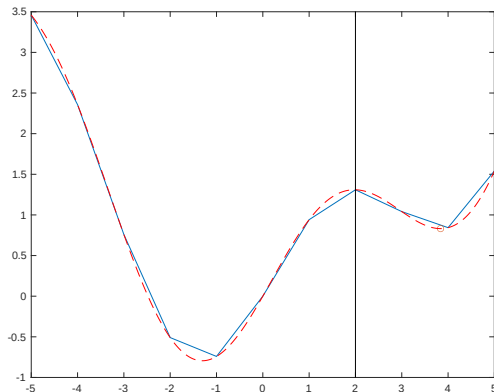


Figure: Solution of a noisy quadratic function with Bounds restrictions

Additional Remarks

Optimization solvers are highly customizable and a wide range of options is available:

- Do you have information about the gradient of the objective? About its Hessian?
- What is the maximum number of iterations before stopping?
- When is the solution precise enough?

→ All those information can be changed. Ex:

```
options = optimoptions('fminunc','Tolfun',1e-12, 'Maxiter', 1e10, 'TolX', 1e-12);  
c0 = 0.15;  
val = @(c) -log(c)-0.98*log(1.02*(0.3-c));  
c = fminunc(val,c0,options)
```

Exercise 2

Solve the following system using the function `linprog()` and plot it.

$$\max_{x,y} f(x,y) = \frac{x}{3} + 2y$$

$$s.t. x \leq 10$$

$$s.t. y \leq 15$$

$$s.t. x + y = 20$$

Assignment

Code and solve the following problem :

$$\min_{K,L} 10K + 5L + 2q$$

$$\text{s.t. } q = 5KL$$

$$\text{s.t. } q \geq 250$$

$$\text{s.t. } K \leq 6$$

$$\text{s.t. } L \geq 0$$

$$\text{s.t. } K_0 = 5$$

$$\text{s.t. } L_0 = 1$$

What is gEconpy?

- Python toolkit for DSGE models, inspired by the R package gEcon.
- Models written in **GCN files** $\Rightarrow$ library handles:
 - FOCs, steady state, linearization
 - IRFs, estimation
- Key idea: stay close to economic blocks (household, firm, shocks).
- Install easily: `pip install gEconpy`

GCN model files: how you write a model

- A .gcn file is a text model with blocks: controls, objective, constraints, identities, calibration, shocks.
- Example (RBC): steady-state identities, household/firm blocks, AR(1) technology shock.
- Big win: calibrate parameters by targeting steady-state relations directly in the calibration block.

Two workflows

- **Exploration/IRFs:**
 - Build a model from GCN
 - Compute steady state and linearization
 - Simulate and plot IRFs
- **Estimation:**
 - Compile the GCN to a state-space model
 - Estimate with PyMC (`statespace_from_gcn`, Kalman filter)

IRFs and plots

- Plotting helpers (`gEconpy.plotting`):
 - `plot_irf`, `plot_simulation`, etc.
- Teaching cue: IRFs = “one shock at a time” analysis.
- Behind the scenes: Blanchard–Kahn conditions ensure existence and uniqueness of solution.

Why show this now?

- Bridges Lecture 2–4 skills (functions, optimization) to **macro models in research**.
- Same structure: objective $\rightarrow$ constraints $\rightarrow$ solver, but in a reproducible `.gcn` model file.
- If you prefer MATLAB: the concepts carry over (state-space, IRFs, linearization).